1) Explain the operations of deep feed forward network with a diagram.

Deep feedforward Networks:

* Deep feedforward networks are also called feedforward neural networks or multilayer Perceptrons. These models are called feedforward because information flows through the function being evaluated from x, through the intermediate computations used to define f and finally to the output y.

* When feedforward neural networks are extended to include feedback connections, they are called recurrent neural network.

Feed forward neural:

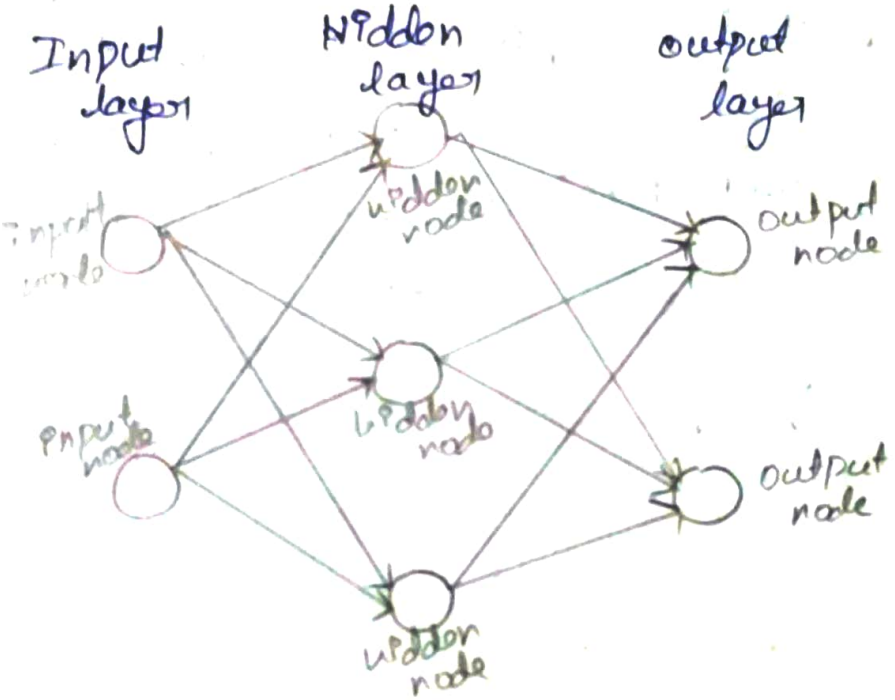
* Feed forward neural network is an artificial neural network in which the connection between nodes does not form a cycle.

* They are called feedforward because information only travels forward in the network (no loop) first through the networks

then through the hidden nodes (if present) and finally through the output nodes.

*Feed forward networks tends to be simple networks that associates inputs with outputs.

Basic structure of feed forward (FF) Neural Network



* Input layer contains one or more input nodes.

* Hidden layer : This layer contains an activation function.

* output layer contains one or more output nodes.

* Feedforward neural networks are primarily used for supervised learning in cases where the data to be learned is neither sequencial nor time-dependent

* Feed forward networks have the following characteristics:

1. Perceptrons are arranged in layers, with the first layer taking in inputs and the last layer producing outputs. the middle layers have no connection with the external world. and hence are called hidden layers.

2. Each perceptron in one layer is connected to every perceptron on the next layer. Hence information is constantly "fed forward" from one layer to the next and this explains why these networks are called feed-forward networks.

3. There is no connection among Perceptrons in the same layer.

Activation function:

* Activation function also known as transfer function are used to map input nodes to output nodes in a certain fashion.

* Activation function helps in normalizing the output between 0 to 1 or -1 to 1.

* Activation function basically decides in any neural network that given input or receiving information is relevant or it is irrelevant.

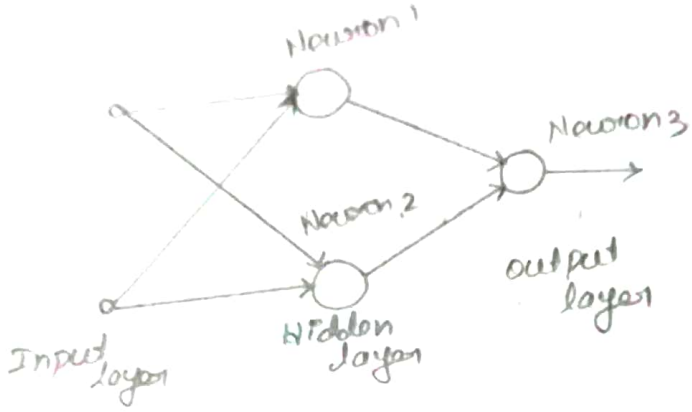
XOR function:

* the XOR function is an operation on two binary values x_1 and x_2. when exactly one of these binary values is equal to 1. the XOR function returns 1. otherwise it returns 0.

* The XOR function provides the target function $y = f'(x)$ that we want to learn.

* Neural networks can be used to classify boolean functions depending on their desired outputs.

* XOR problem is not linearly separable.



TOP naural labelled as "Neuron 1" in the hidden layer.

$$w_{11} = w_{12} = +1$$

Slope of the decision boundary constructed by this hidden neuron is equal to -1.

The bottom neuron labelled as "Neuron 2" in the hidden layer.

$$w_{21} = w_{22} = +1$$

Output neuron labelled as "Neuron 3"

$$w_{31} = -2, w_{32} = +1$$

2)

* Each neuron is represented by a McCulloch-pitts model, which uses a threshold function for its activation function.

* Bits 0 and 1 are represented by the levels 0 and +1 respectively.

* Function of the output neuron is to construct a linear combination of the decision boundaries formed by the two hidden neurons.

* when both hidden neurons are off, which occurs when the input neurons is (0,0) the output neuron remains off when both hidden neurons are on, which occurs when the input pattern is (1,1).

* when the top hidden neuron is off and bottom hidden neuron is on which occurs when the input pattern is (0,1) or (1,0).

2) Analyze the types of non-linearity in a convolutional neural network and explain in detail.

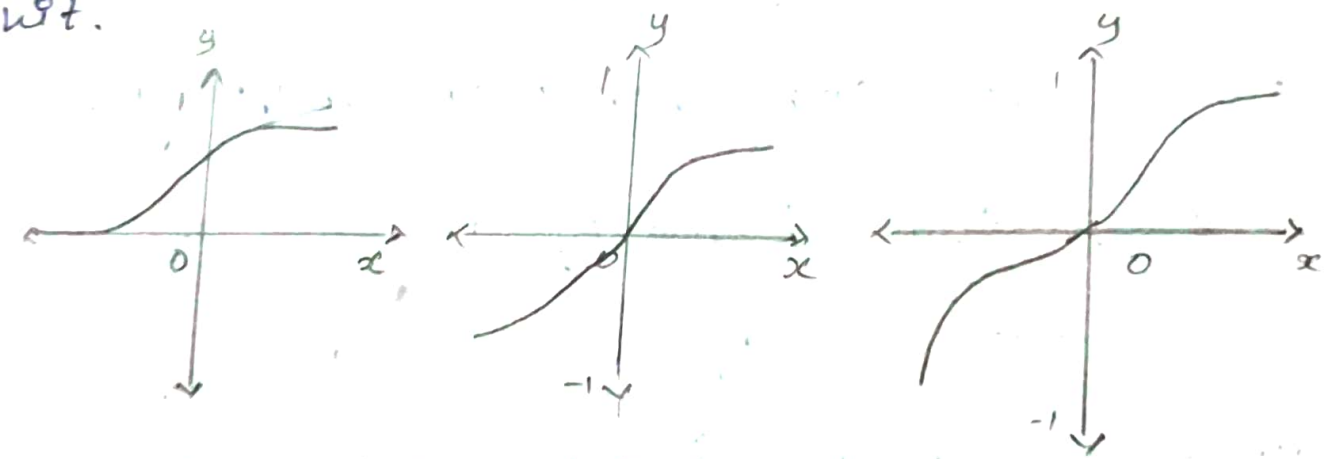
CNN learning : Nonlinearity functions

* The weight of layers in a CNN are often followed by a nonlinear activation function. The activation function takes a real valued input and squashes it within a small range such as [0; 1] and [-1; 1]. The application of a nonlinear function after the weight layers is highly important. since it allows a neural network to learn nonlinear mappings.

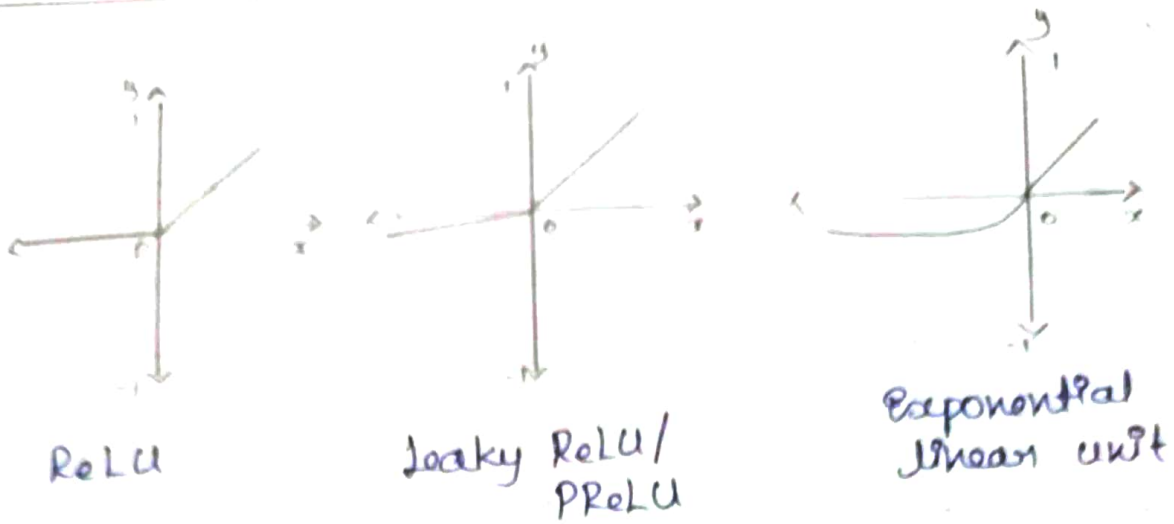
* In the absence of nonlinearities, a stacked network of weight layers is equivalent to a linear mapping from the input domain to the output domain.

* A nonlinear function can also be understood as a switching or a selection mechanism which decides whether a neuron will fire or not given all of its inputs the activation function that are commonly used in deep networks are differentiable to enable error back propogation.

* common activation functions that are used in deep neural networks are Sigmoid, Tanth, Algebraic, Sigmoid, ReLU, Leaky RELU/PReLU and exponential linear unit.



Sigmoid Tanth Algebraic Sigmoid



ReLU | Leaky ReLU/ PReLU | Exponential linear unit

Sigmoid:

The sigmoid activation function takes in a real number as its input and outputs a number in the range of [0,1]

Tanh:

The Tanh activation function implements the hyperbolic tangent function to squash the input values within the range of [-1;1].

Algebraic sigmoid function:

The algebraic sigmoid function also maps the input within the range [-1;1].

Rectifier linear unit

The ReLU is a simple activation function which is of a special practical importance because of its quick computation. A ReLU function maps the input to a 0 if it is negative and keeps its value unchanged if it is positive.

Leaky ReLU:

The leaky ReLU function completely switches off the output if the input is negative. A leaky ReLU function does not reduce the output to a zero value, rather it outputs a down-scaled version of the negative input.

Exponential linear units:

The exponential linear units have both positive and negative values and they therefore try to push the mean activation toward zero. It helps in speeding up the training process which achieving a better performance.

convolutional operation

* convolutional operation focuses on extracting / preserving important features from the input: convolutional operation allows the network to detect horizontal and vertical edges of an image.

* In general form convolutional is an operation on two functions of a real valued argument.

* Suppose we are tracking the location of a spaceship with a laser sensor. Laser sensor provides a single output $x(t)$.

* Now suppse that our laser sensor is somewhat noisy. To obtain a less noisy estimate of the spaceship's position.

* we can do this with a weighting function $w(a)$, where a is a age of the measurement.

* Convolutional operation uses three Parameters Input image, feature detector and feature map.

* It involves an input matrix and a filter also known as kernal. Input matrix can be pixel values of a grayscale image.

* Input image is converted into binary 1 and 0. Input to convolutional can be raw data or a feature map output from another convolutional.

* Sometimes a 5×5 or 7×7 matrix is used a feature detector. Feature detector is often referred to as a "kernal" or a "filter" At each step the kernal is multiplied by the input data values within its bounds, creating a single entry in the output feature map.

3) Explain gradient computation in RNN and discuss the vanishing and exploding gradient problems.

Gradient computation:

It is simple to calculate the gradient using a recurrent neural network. The unrolled computational network is simply subjected to the generalized back-propagation method.

* The gradient with respect to each parameter must be summed across all places that the parameter occurs in the unrolled net.

In simplified model, we denote h_t as the hidden state x_t as input and o_t as output at time.

$$h_t = f(x_t, h_{t-1}, w_h).$$

$$o_t = g(h_t, w_o)$$

when f and g are transformations of the hidden layer and output layer respectively

$$L(x_1, \ldots x_T, y_1, \ldots y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1} l(y_t, o_t)$$

$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^{t} \frac{\partial l(y_t, o_t)}{\partial w_h}$$

$$= \frac{1}{T} \sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$$

It is complex for backpropogation espcially when we compute the gradients with regard to the parameters w_h of the objective function L.

$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial w_h}$$

$$= \frac{1}{T} \sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_c)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$$

$$a_t = b_t + \sum_{i=1}^{t=1} \left(\prod_{j=i+1}^{t} c_j \right) b_i$$

By substituting a_t, b_t and c_t

$$a_t = \frac{\partial h_t}{\partial w_h}$$

$$b_t = \frac{\partial f(x_t, h_{t-1}, w_n)}{\partial w_n}$$

$$c_t = \frac{\partial f(x_t, h_{t-1}, w_n)}{\partial h_{t-1}}$$

The gradient computation satisfies

$$a_t = b_t + c_t a_{(t-1)}$$

1. Vanishing Gradient problem

The vanishing gradient occurs when gradients become progressively smaller as they are propogated backward through many time steps.

For example, if each step multiplies the gradient by a value smaller than 1.

$0.5 \times 0.5 \times 0.5 \times \ldots$

the gradient quickly approaches zero

2. Exploding Gradient problem

The exploding gradient occurs when gradients become extremely large during backpropogation.

If the repeated multiplication involves values greater than 1.

$2 \times 2 \times 2 \times \ldots$

the gradient can grow rapidly.